DAILY CLIMATE FORCASTING

Shirin Malekpour

1.PROJECT DESCRIPTION

In this project, you are tasked with developing predictive models to forecast atmospheric pressure using historical climate data from the city of Delhi. The objective of this project is to apply and enhance the machine learning techniques you have learned throughout the course to achieve accurate predictions of mean atmospheric pressure.

2.DATA-SET DESCRIPTION

CONTENT

The Dataset is fully dedicated for the developers who want to train the model on Weather Forecasting for Indian climate. This dataset provides data from **1st January 2013** to **24th April 2017** in the city of Delhi, India. The 4 parameters here are
meantemp, humidity, wind_speed, meanpressure.

ACKNOWLEDGEMENTS

This dataset has been collected from Weather Underground API. Dataset ownership and credit goes to them.

3.IMPORTING LIBRARIES

```
import pandas as pd
import matplotlib as plt
import seaborn as sns
import numpy as np
import datetime as dt

import plotly
import plotly.express as px
import plotly.graph_objects as go
import cufflinks as cf
```

4.EXPLORATORY DATA ANALYSIS

1.IMPORTING DATA:

```
df_test=pd.read_csv('DailyDelhiClimateTest.csv',parse_dates=['date'], index_col="date")

df_train=pd.read_csv('DailyDelhiClimateTrain.csv',parse_dates=['date'], index_col="date")
```

I imported the two train and test sets using pandas and then using `parse_dates=['date']` tell Pandas to parse the date column as a datetime object. This means that the values in the date column will be converted from strings to datetime objects, which will be useful for date-based operations and analysis.

index_col="date" sets the date column as the index of the DataFrame. This means that the date column will become the row labels of the DataFrame, and Pandas will use these dates to identify and access specific rows.

2.DATA EXPLORATION:

As you can see there are no null values and there are 4 features to be considered

```
#   Column        Non-Null Count  Dtype
---  ------        --------------  -----
0   meantemp      114 non-null    float64
1   humidity      114 non-null    float64
2   wind_speed    114 non-null    float64
3   meanpressure  114 non-null    float64
dtypes: float64(4)
```

train:

```
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   meantemp      1462 non-null   float64
 1   humidity      1462 non-null   float64
 2   wind_speed    1462 non-null   float64
 3   meanpressure  1462 non-null   float64
dtypes: float64(4)
```

To strike a balance, I created a third dataset, called "full_df," which is a concatenation of train and test data. I Use this full dataset for EDA and feature engineering, but make sure to split it back into train and test datasets before training my model.

```
RangeIndex: 1576 entries, 0 to 1575
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   meantemp      1576 non-null   float64
 1   humidity      1576 non-null   float64
 2   wind_speed    1576 non-null   float64
 3   meanpressure  1576 non-null   float64
dtypes: float64(4)
memory usage: 49.4 KB
```

3.FEATURE ENGINEARING

I've created some new features by breaking down the datetime index into its constituent parts (year, month, day, day of the week) for better visualization over time and analyzing data from different angles.

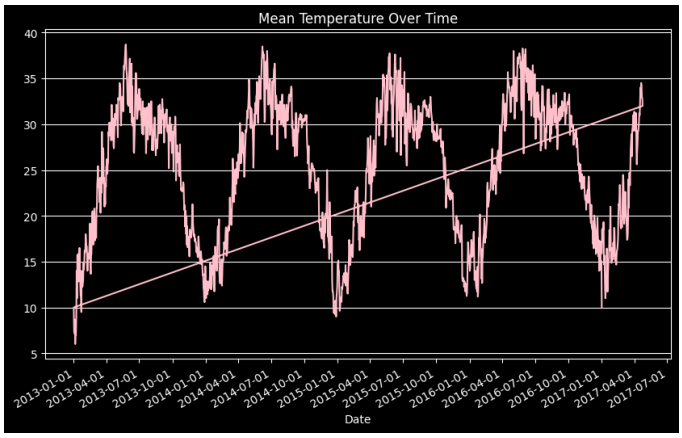
Sample output:

	meantemp	humidity	wind_speed	meanpressure	year	month	day	dayofweek	date
date									
2016-02-02	15.266667	55.866667	8.893333	1018.2	2016	2	2	1	2016-02-02

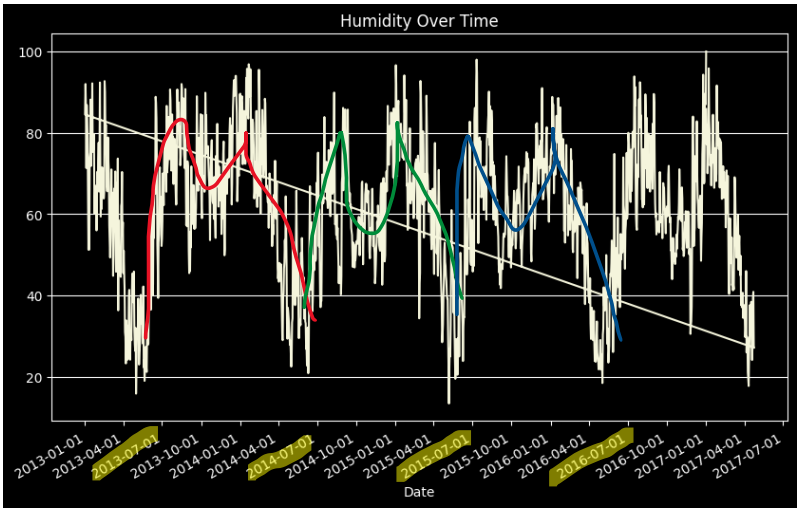
5.VISUALISATION AND ANALYSIS

Let's graph each attribute against time to understand the patterns better:

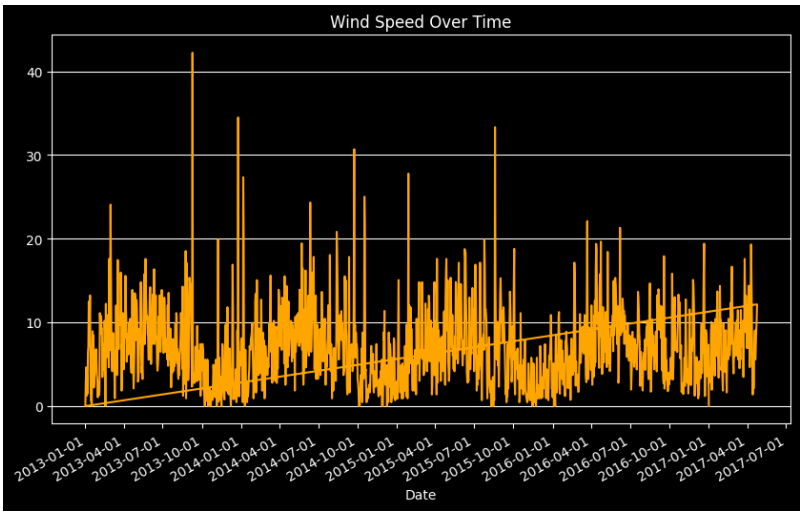
1. **Mean temperature over time**: The graph displays mean temperature data over a period from 2013 to 2017. There is a Seasonal Variations: The graph shows periodic variations, likely corresponding to seasonal changes. For example, there are peaks and valleys that repeat annually. The vertical axis represents temperature values, ranging from 5 to 40



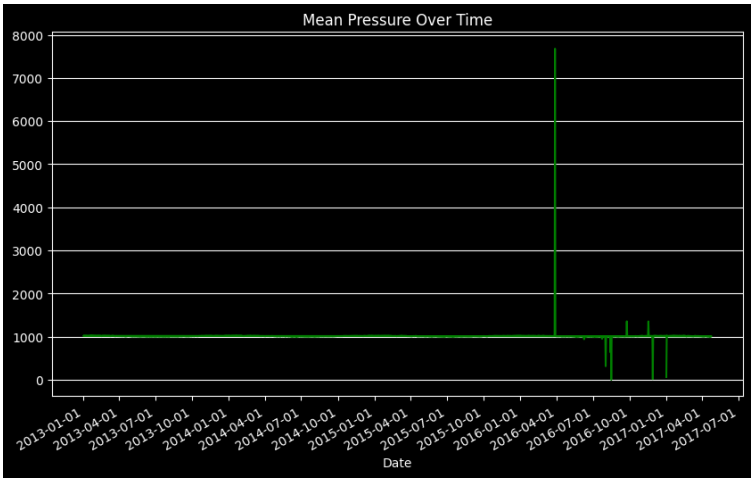
2. Humidity over time: The graph displays mean temperature data over a period from 2013 to 2017. It is more fluctuated that the temperature graph but it still shows a seasonal trend (the humidity is most around the beginning of autumn and repeats annually.)



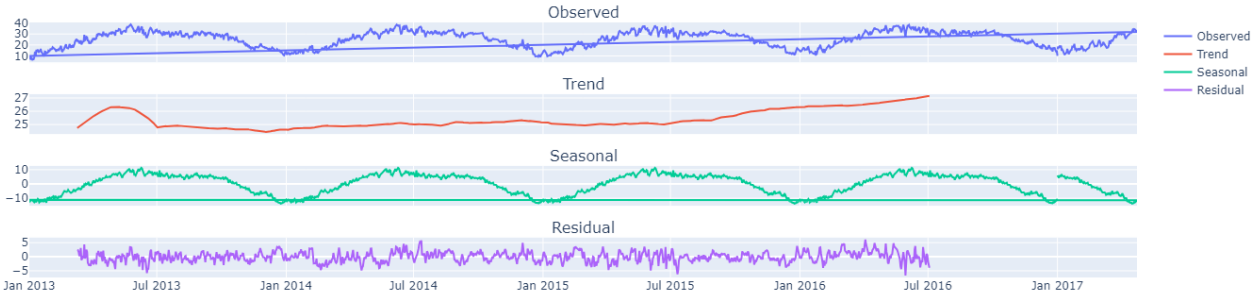
3. The same seasonal trend is also true for wind speed graph:
Several spikes in wind speed are visible. These spikes represent moments of high wind speeds. The vertical lines seem to indicate variability or measurement error at each data point. there isn't a clear overall trend. Wind speed appears to vary without a consistent upward or downward pattern.



4. Mean pressure over time:
   Pressure Spikes: The graph exhibits several spikes in pressure at different points in time. The most significant spike occurs between 2015 and 2016, where the pressure value sharply peaks at approximately 7000.
   Overall Trend: Apart from the spikes, the rest of the graph shows relatively low-pressure values with minor fluctuations. There seems to be no consistent upward or downward trend.

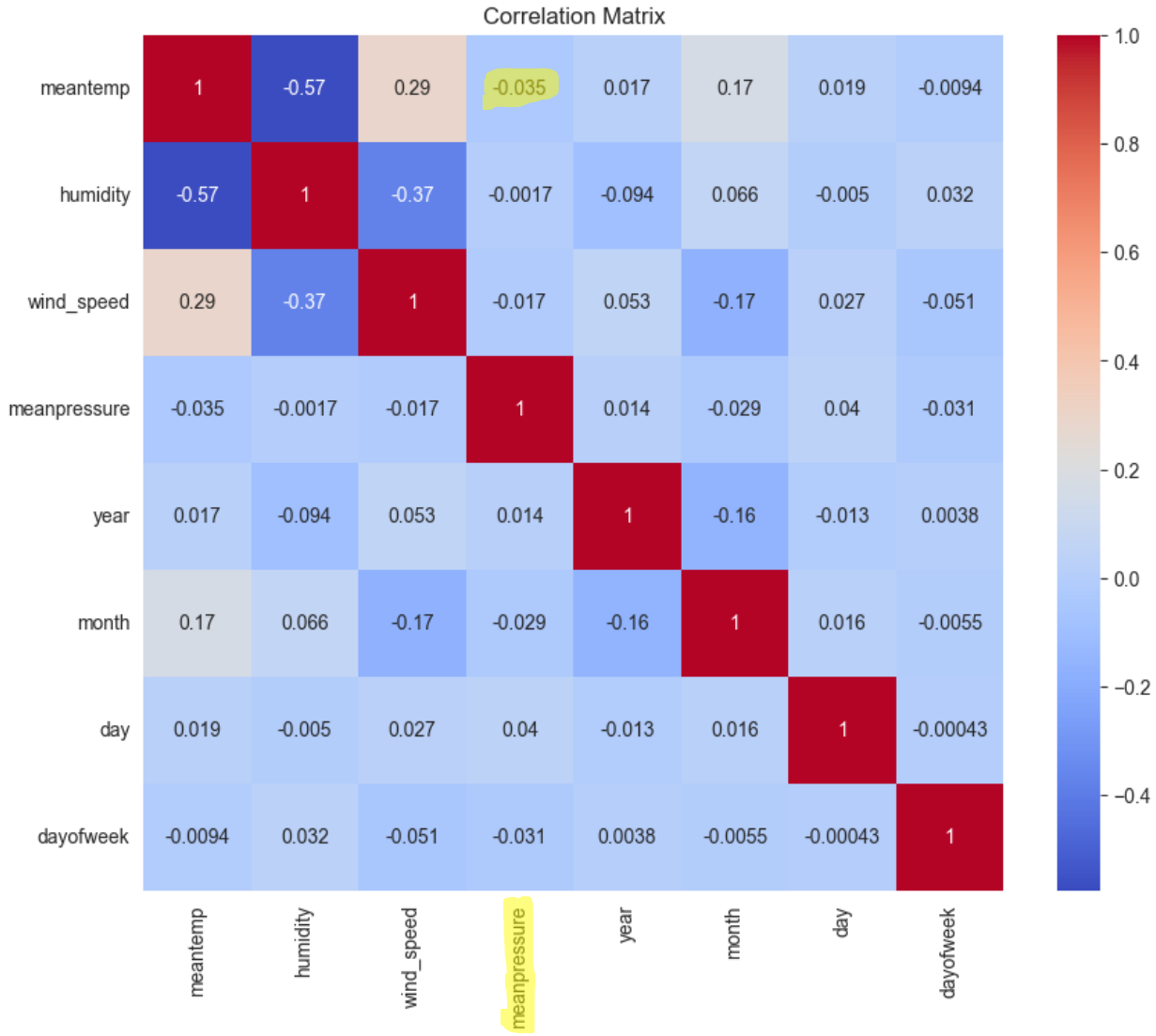


5. BREAKING DOWN THE DATA INTO SEASONAL, TREND AND RESIDUAL COMPONENTS:



1. **Observed Component**:
    - The blue line represents the observed data. It fluctuates around the 30-value mark over time from January 2013 to January 2017.
    - This component includes all effects: trend, seasonality, and irregularities.
2. **Trend Component**:
    - The red line depicts the trend. It gently rises over the same period, indicating an overall increase in the data's trend.
    - Trends capture long-term changes or patterns.
3. **Seasonal Component**:
    - The green line oscillates above and below the zero line. It suggests regular seasonal variations in the data.
    - Seasonality represents recurring patterns within shorter time intervals (e.g., yearly, monthly, weekly).
4. **Residual Component**:
    - The purple line in the "Residual" plot shows random fluctuations around the zero line.
    - Residuals represent irregular components or noise in the data after accounting for trend and seasonality.

6.CORRELATION AND HEAT MAP:



Meantemp and Humidity:

1. There's **a <u>relatively strong negative correlation between "meantemp" (mean temperature) and "humidity."</u>** As temperature increases, humidity tends to decrease or vice versa. the correlation coefficient is approximately -0.57.

2. Wind Speed and Meanpressure:

   There's **a <u>weak positive correlation between "wind_speed" and "meanpressure."</u>** <u>the</u> correlation coefficient is approximately 0.29.

3. Year and Month:

   There's a positive correlation between "year" and "month." the correlation coefficient is approximately 0.17.

4. Day of Week and Meanpressure:

There's a very weak negative correlation between "day_of_week" and "meanpressure." the correlation coefficient is approximately -0.0094.

5.FEATURE SELECTION:

Conclusion: between all the attributes mean pressure has the lowest correlation with mean temperature plus it has a lot of insignificant spikes and fluctuations so I won't consider this attribute in my model training

```
DatetimeIndex: 1576 entries, 2017-01-01 to 2017-01-01
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   meantemp    1576 non-null   float64
 1   humidity    1576 non-null   float64
 2   wind_speed  1576 non-null   float64
dtypes: float64(3)
memory usage: 49.2 KB
```

6.DATA PREPROCESSING FOR MODELING

1. test\train split:

I did a 80/20 % split for train and test .

```
len(train)
✓ 0.0s
1261

len(test)
✓ 0.0s
315
```

2.scaling:

For RNN, LSTM, and GRU:

These models often benefit from scaling, especially if you are using activation functions like sigmoid or tanh, which are sensitive to the scale of the input data. Scaling helps stabilize training by putting all features on a similar scale, which can speed up convergence and prevent exploding gradients.

The data will be between 0 and 1:

```
array([[0.45851528, 0.44932432, 0.36018813],
       [0.47161572, 0.44003378, 0.23177912],
       [0.47953057, 0.52776605, 0.05788134],
       [0.50627729, 0.52111486, 0.12642113],
       [0.51353712, 0.49864865, 0.05873993],
       [0.51353712, 0.53885135, 0.07895152],
       [0.51965066, 0.58688767, 0.08793226],
```

7.MODEL TRAINING:

RNN

RNN architecture:

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 12, 128)	16,896
dropout (Dropout)	(None, 12, 128)	0
simple_rnn_1 (SimpleRNN)	(None, 12, 64)	12,352
simple_rnn_2 (SimpleRNN)	(None, 32)	3,104
dense (Dense)	(None, 64)	2,112
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 111,269 (434.65 KB)

Trainable params: 37,089 (144.88 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 74,180 (289.77 KB)

SimpleRNN Layer:

- Output Shape: (None, 128)
- Parameters: 16,896
- This layer is a basic RNN cell with 128 units

Dropout Layer (after SimpleRNN):

- Output Shape: (None, 128)
- Dropout is applied to prevent overfitting.

Dense Layer:

- Output Shape: (None, 64)
- Parameters: 8,256
- A fully connected layer with 64 units.

Dropout Layer (after Dense):

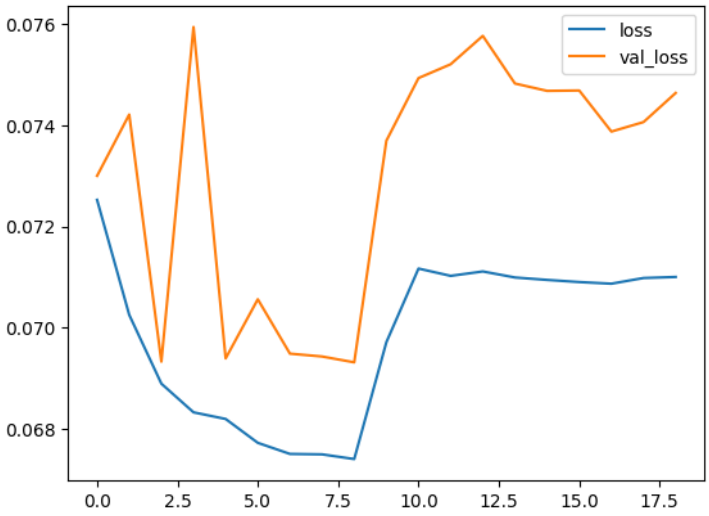
- Output Shape: (None, 64)
- Another dropout layer for regularization.

Dense Layer (Output Layer):

- Output Shape: (None, 32)
- Parameters: 2,080
- Final layer with 32 units.

Dropout Layer (after Output Layer):

- Output Shape: (None, 16)
- Final dropout layer.



The plot shows the training and validation loss curves, as well as the training and validation accuracy curves, over the course of the training process. A decreasing validation loss indicates that the model's performance on the validation set is getting better, suggesting that it is becoming more effective at generalizing to new data.

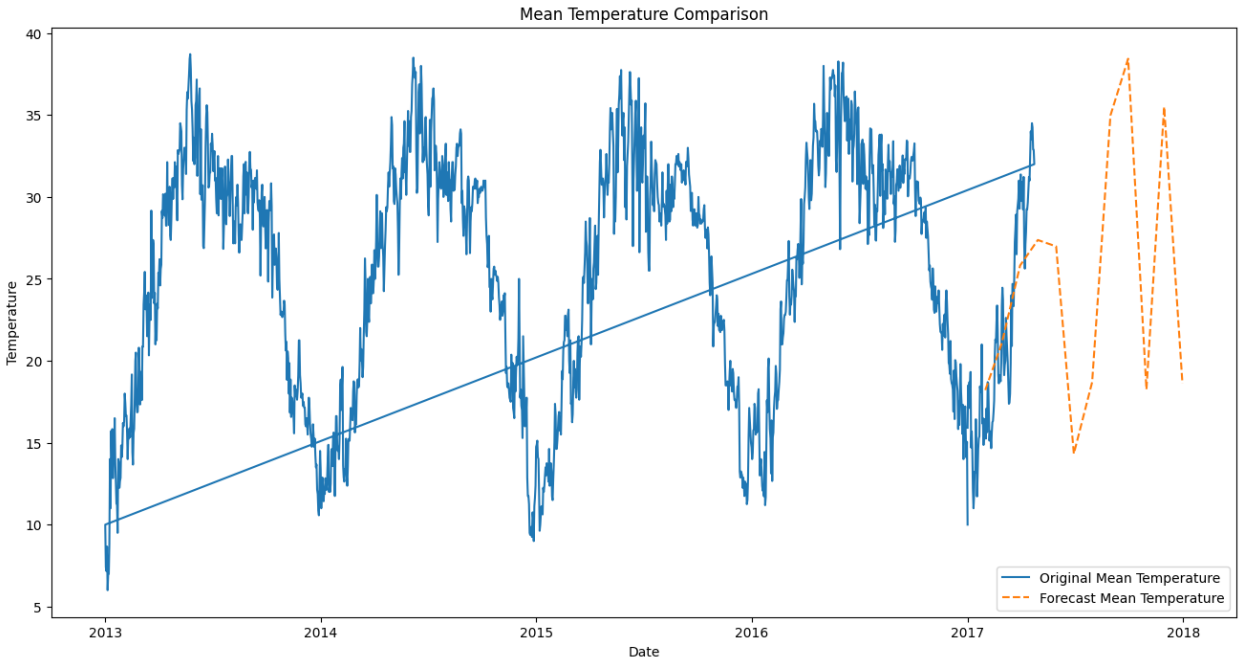
EVALUATION METRICS:

```
mae: 0.07136626788154794

mse: 0.009314435210961371

rmse: 0.09651132167244095
```

And here is the forcasted results on the graph:

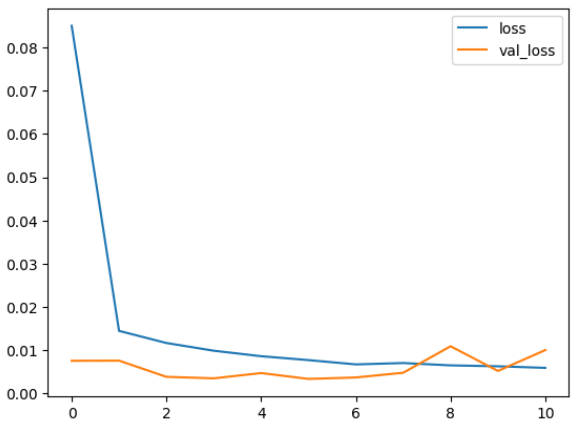


LSTM

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) architecture. LSTMs are designed to address the vanishing gradient problem present in traditional RNNs. They excel at capturing long-term dependencies, making them ideal for sequence prediction tasks.

Here I have used a bidirectional LSTM: Bidirectional LSTMs are an extension of traditional LSTMs that can improve model performance on sequence classification problems.In problems where all timesteps of the input sequence are available, Bidirectional LSTMs train two instead of one LSTMs on the input sequence. The first on the input sequence as-is and the second on a reversed copy of the input sequence. This can provide additional context to the network and result in faster and even fuller learning on the problem.

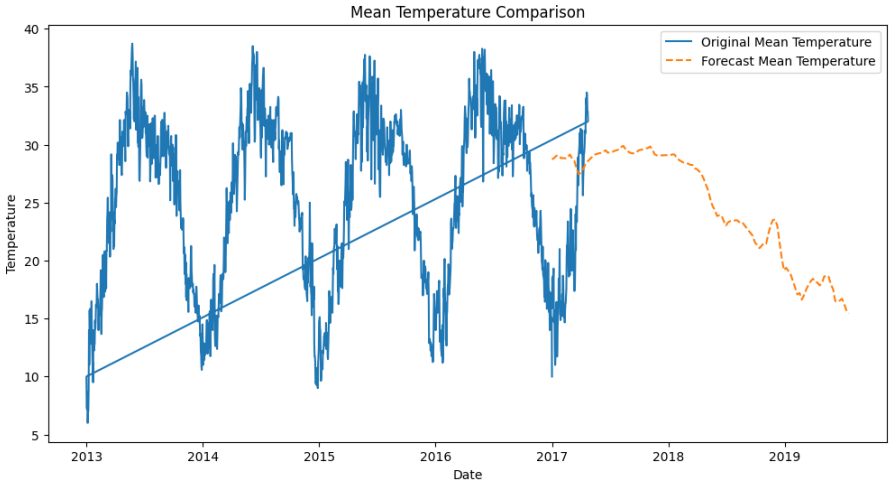
Training vs validation loss graph:



EVALUATION METRICS(LSTM):

```
mae: 0.04648763742659955
mse: 0.003370400729716507
rmse: 0.05805515248207093
```

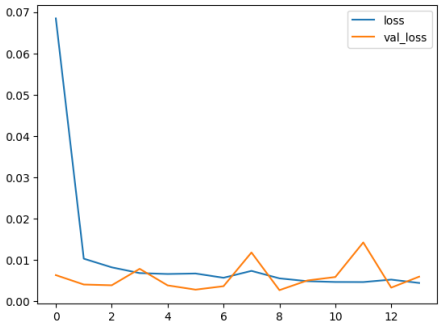
And here is the forcasted results on the graph:



GRU

- **Gated Recurrent Unit (GRU)** is a type of recurrent neural network (RNN) architecture introduced as an alternative to Long Short-Term Memory (LSTM) networks. Simpler than LSTMs (fewer parameters).

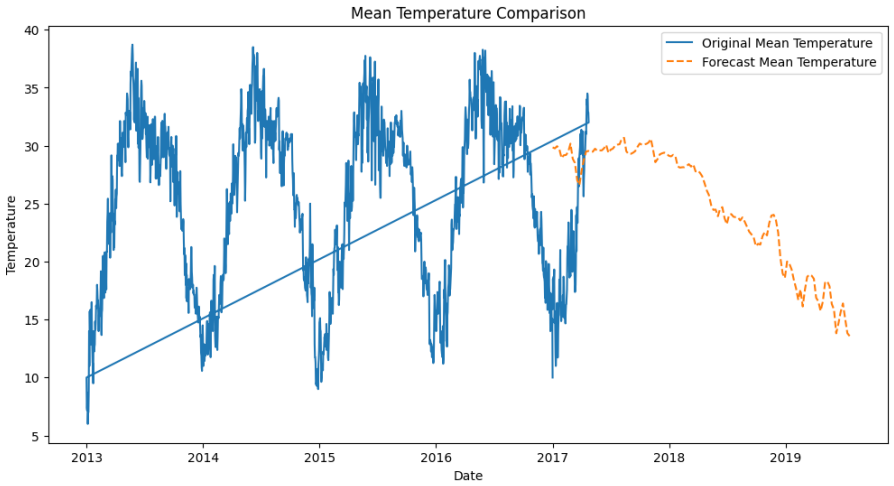
Training vs val loss graph:



EVALUATION METRICS:

```
mae: 0.042782766529532404
mse: 0.002699328225246699
rmse: 0.051955059669359434
```

And here is the forcasted results on the graph:



8.Conclusion:

Lowest mean square error: GRU< LSTM<RNN

Lowest mean average error: GRU (but LSTM is very close) <RNN

Lowest root mean square error: GRU (but LSTM is very close) <RNN